Minigestor de Base de Datos: Parte 1 y Parte 2

Sebastian Loli Gonzalez
Tamy Kristel Flores Guevara
Fabricio Bautista
Diego Roger Illescas Castillo
Badi Masud Rodriguez Ramirez

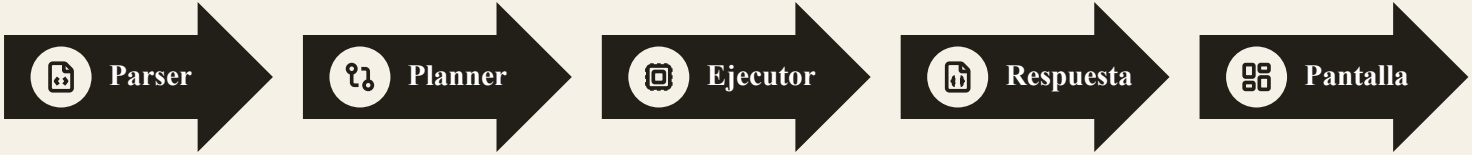
Arquitectura en cuatro capas

L1	**Frontend · React 19 + Vite**	4 paneles más el mapa Leaflet
L2	**API REST · FastAPI**	/api/query, /api/tables, import CSV
L3	**Motor · Python estándar**	parser, planner por reglas, ejecutor
L4	**Almacenamiento e índices**	Heap File, Secuencial, B+, Hash, R-Tree

Sin dependencias

El motor usa solo la librería estándar de Python.

Del texto SQL al resultado



sql_parser.py construye el AST desde POST /api/query.

Elige entre Hash, B+, R-Tree o Heap Scan, y lo justifica.

Resuelve el Catalog, lee de disco y aplica índices y operadores.

columns, rows, tiempos y execution_plan.

React pinta resultados y la traza real con EXPLAIN ANALYZE.

Parte 1 por dentro

Estructura	Cómo funciona por dentro	Consecuencia
Heap File	Páginas de 4 096 B, 95 slots de 43 B, RID = (página, slot); una free-list reutiliza los huecos	Insertar y borrar es barato; buscar por clave recorre todo
Archivo Secuencial	.main ordenado más .aux para las altas; se reorganiza al pasar el 30 % de desperdicio	Búsqueda binaria; al reorganizar cambian los RID y se reconstruyen los índices
B+ (orden configurable; 64 en los benchmarks)	Hojas enlazadas; en el agrupado la hoja guarda el registro, en el no agrupado guarda (clave, RID)	Buen rango; el no agrupado paga una lectura por RID
Hash extendible	Directorio de 2^d entradas y buckets de capacidad fija (64 en los benchmarks) ; si un bucket se llena se divide y, si hace falta, se duplica el directorio	Igualdad exacta rápida; no sirve para rangos ni ORDER BY

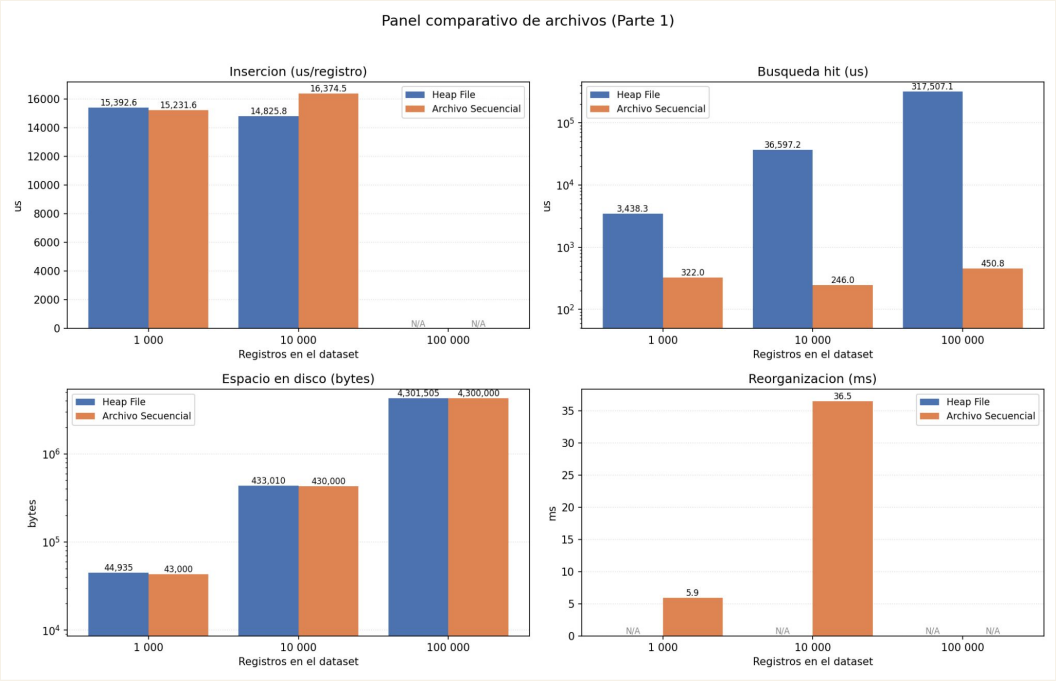
Los tres índices y los dos archivos se diferencian en una cosa: qué guarda cada hoja o página. Eso explica buena parte de los resultados que vimos. Por ejemplo, al reorganizar el Secuencial todos los RID cambian, así que el motor reconstruye los índices.

Dos formas de guardar en disco

Dos formas de guardar en disco

Criterio	Heap File	Archivo Secuencial Paginado
Orden físico	Orden de llegada	Ordenado por la clave
Inserción (1 000)	15 392 µs	15 231 µs
Búsqueda por clave (100 000)	317 507 µs	450 µs
Reorganización (10 000)	No aplica	36,5 ms al pasar del 30 %
Cuándo conviene	Altas y bajas constantes	Consultas por clave primaria

Heap vs Secuencial, medido



Inserción, búsqueda por clave, espacio en disco y reorganización

Tres índices, tres reglas

Cada índice y la consulta que resuelve mejor

Índice	Mejor operación	Lo que cuesta
Hash Extendible	Igualdad exacta, muchas lecturas	No sirve para rangos ni orden
B+ no agrupado	Igualdad y rango con índice compacto	Una lectura por RID en cada fila
B+ agrupado	Rango cuando hay que traer la fila y ORDER BY	1,9× más espacio; hoja pesada

Los índices, medidos a 50 000 claves

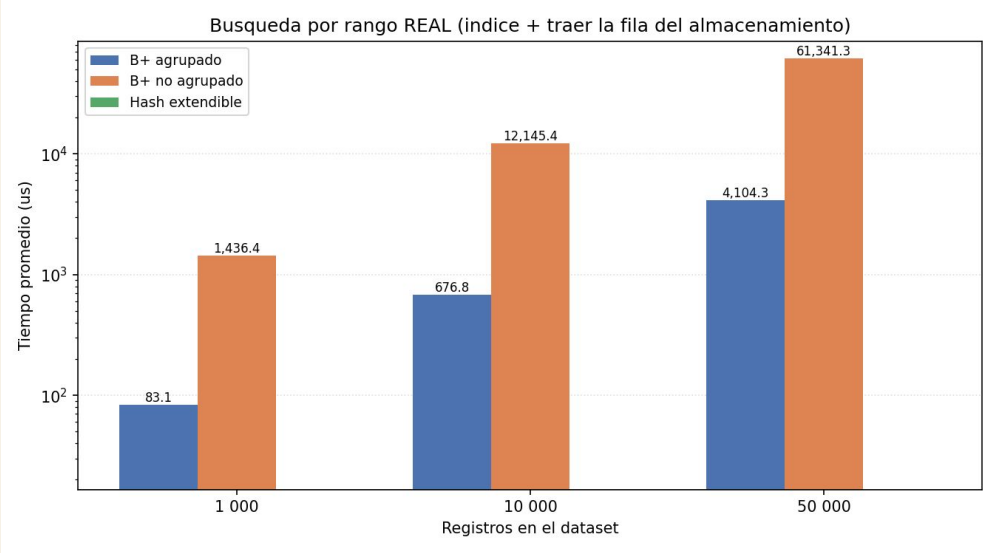
Igualdad, rango y espacio a 50 000 claves

Índice	Construcción (ms)	Igualdad (µs)	Rango (µs)	Bytes/registro
Hash extendible	315	**1,4**	—	24,7
B+ no agrupado	860	**2,0**	380,8	25,3
B+ agrupado	1 334	**11,0**	3 681,7	45,7
Sin índice (scan)	—	**25 509,1**	—	0

benchmark_results/index_benchmark.json · semilla 42 · 500 igualdades y 100 rangos

Tiempo por construcción (promedio de 3 repeticiones)

La comparación justa del rango



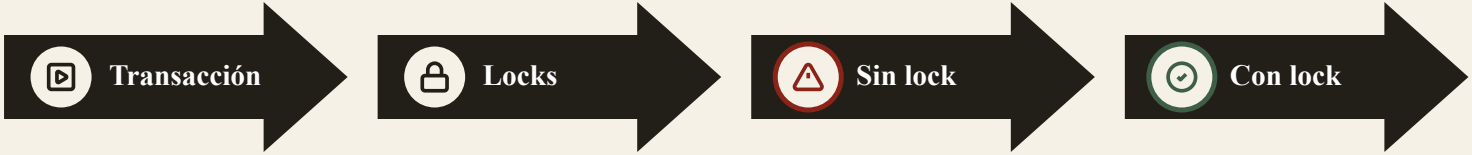
Rango con índice más recuperación de la fila: 4 104 µs (agrupado) contra 61 341 µs (no agrupado) a 50 000

Ordenar y unir sin caber en memoria

Los tres algoritmos externos y cuándo entra cada uno

	Algoritmo	Para qué	Cómo lo hace el motor
⇅	**External Sort**	ORDER BY sin índice que lo cubra	Genera runs ordenados y los une con k-way merge
⇄	**External Hashing**	GROUP BY y equi-JOIN grandes	Particiona en disco y resuelve por partición
⚡	**Uso del índice**	ORDER BY cubierto por un B+ (agrupado o no agrupado)	El planner evita el External Sort por completo

Concurrencia sin perder datos



BEGIN TRANSACTION agrupa las operaciones y END confirma (COMMIT); ROLLBACK revierte.

El LockManager concede bloqueos y serializa el conflicto.

Dos hilos cruzan sus lecturas y se pierde una actualización.

Las transacciones se serializan y el valor siempre es el esperado.

La interfaz de cuatro paneles

Los cuatro paneles del enunciado, más el mapa

2.1.5

Panel de archivos

Tablas cargadas, esquema, índices y tamaño en disco.

2.1.5

Panel de consultas

Editor SQL con ejemplos y ejecución.

2.1.5

Panel de resultados

Tabla con las filas y el tiempo de ejecución.

2.1.5

Panel de plan

Índices usados, orden de operadores y la razón.

2.2.2

Panel de mapa

Puntos, radio y polígono resaltados sobre Leaflet.

La teoría, medida

Lo que dice la clase y lo que mide el benchmark

Regla del curso	Cómo se cumple en el proyecto	Medición
La hoja del B+ agrupado lleva el registro	Nodo interno igual; hoja con datos	39 000 de 44 773 B son registros
El no agrupado paga un acceso por RID	Recorre igual, pero vuelve a la tabla	4 104 µs contra 61 341 µs: 15×
MINDIST es cota inferior, no distancia	Acota y poda; la distancia real aparte	Rango, k-NN y polígono exactos
Las semillas se eligen por área	PickSeeds maximiza el área desperdiciada	MINDIST solo poda, no decide el split
Los MBR pueden solaparse	No se fuerza partición disjunta	validate() comprueba las invariantes

El benchmark contradecía la teoría del B+ agrupado: se revisó la medición y estaba mezclando el recorrido del índice con la recuperación de la fila

ISSUE 02

Parte 2 · El motor espacial

Separador: índice R-Tree, SQL espacial y comparativa con PostGIS.

R-Tree: podar, no aproximar

El R-Tree: qué hace cada pieza

Pieza	Qué es	Para qué sirve en la consulta
MBR	Rectángulo mínimo que encierra a los hijos	Podar ramas enteras sin bajar a sus hojas
MINDIST	Distancia mínima del punto al MBR	Da una cota inferior exacta: si ya perdí, no bajo
Split cuadrático	Elige la partición más barata al desbordar	Mantiene los MBR pequeños y poco solapados
k-NN best-first	Cola de prioridad por MINDIST	Saca el vecino más cercano primero, sin recorrer todo
Ambas métricas	Euclidiana y Haversine geodésica	Radio en metros sobre latitud y longitud

SQL espacial: la sintaxis nueva

```
CREATE INDEX idx_tiendas_ubicacion ON tiendas (lat, lon) USING RTREE;

SELECT * FROM tiendas
WHERE distancia(lat, POINT(-12.0464, -77.0428)) < 5000;

SELECT nombre, lat, lon FROM restaurantes
ORDER BY distancia(lat, POINT(-12.0464, -77.0428)) LIMIT 10;

SELECT * FROM tiendas
WHERE dentro_de(lat, POLYGON((-12.20 -77.20, -12.20 -76.90,
                             -12.00 -76.90, -12.00 -77.20)));
```

Las tres consultas espaciales

Consulta	Qué responde	Plan que elige el optimizador
Por rango	Puntos dentro de un radio	RTREE_RANGE_SCAN
k-NN	Los k vecinos más cercanos	RTREE_KNN
Intersección	Puntos dentro de un polígono	RTREE_POLYGON

- [ ] Rango y k-NN podan con MINDIST
- [ ] El polígono poda con su MBR y usa ray casting
- [ ] Los tres son exactos: el R-Tree poda, no aproxima

Secuencial vs R-Tree vs PostGIS

100 000 puntos · promedio de 100 consultas por medición

Consulta	Secuencial	R-Tree propio	GiST de PostGIS	R-Tree vs secuencial
Rango 1 km	88,55 ms	2,06 ms	0,34 ms	**43,1×**
Rango 5 km	88,85 ms	11,63 ms	10,03 ms	**7,6×**
Rango 10 km	89,17 ms	38,09 ms	43,49 ms	**2,3×**
k-NN con k = 10	116,29 ms	11,35 ms	0,26 ms	**10,2×**
k-NN con k = 100	174,90 ms	12,95 ms	0,63 ms	**13,5×**
Polígono de 4 vértices	—	11,13 ms	—	**—**

benchmark_results/spatial_benchmark.json · PostgreSQL 16.15 con PostGIS 3.4.2 · semilla 42

Con 1 000 puntos, el R-Tree propio gana a PostGIS en rango (4,3× en 1 km)

Limitaciones

- No hay WAL ni recuperación ante fallos; la atomicidad se logra restaurando el estado previo en memoria.
- Una sola transacción activa; bloqueos por tabla con timeout de 2 s y sin detección de interbloqueos.
- Los índices (incluido el R-Tree) viven en memoria y se reconstruyen al abrir.
- Los tiempos de la Parte 1 están inflados por el entorno (Windows); comparamos solo en relativo.

Listo para la Demo

Cierre: alcance entregado, rigor de las mediciones y aprendizajes.